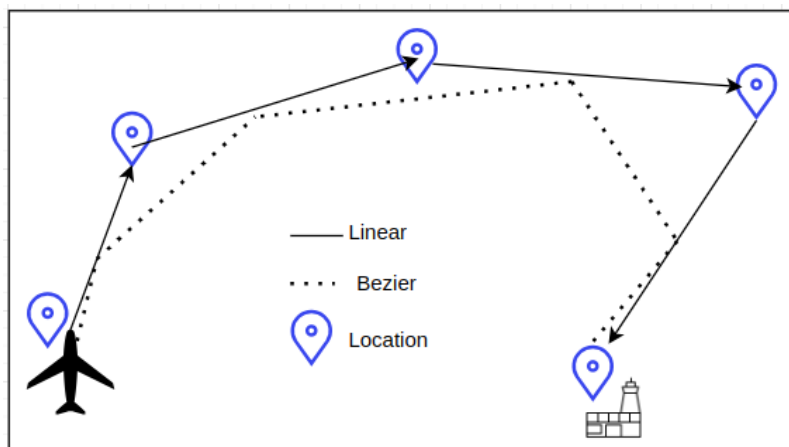


Ejercicio 1

La clase Route representa una ruta que está compuesta por al menos dos coordenadas. Las coordenadas son instancias de la clase Location. En el siguiente diagrama de clases se presenta una versión simplificada de Route que implementa las operaciones:

Route	
+ waitPoints: Location [2..n] + currentIdx: Integer	
+ <u>Route(coordinates: Location[]): Route</u> + next(): Location + fullDistance(param: String): Real + distanceLeft(param: String): Real	next: retorna el siguiente punto del recorrido. fullDistance: tiene como parámetro un String que denota un tipo de interpolación (Linear o Bezier). Esta operación retorna la distancia total del recorrido según el parámetro, que puede tener valores "Linear" o "Bezier". distanceLeft: tiene como parámetro un String que denota un tipo de interpolación (Linear o Bezier). Esta operación retorna la distancia que falta recorrer según el parámetro, que puede tener valores "Linear" o "Bezier".



1	public double fullDistance(String param) {
2	double res = 0;
3	for (int i = 1; i < waitPoints.length; i++) {
4	if (param.equals("Linear"))
5	res += waitPoints[i].distanceTo(waitPoints[i-1]);
6	else
7	res += (waitPoints[i].middle(waitPoints[i-1])).distanceTo(waitPoints[i-1]);
8	}
9	return res;
10	}

11	public double distanceLeft(String param) {
12	double res = 0;
13	for (int i = currentIdx + 1; i < waitPoints.length; i++) {
14	if (param.equals("Linear"))
15	res += waitPoints[i].distanceTo(waitPoints[i-1]);
16	else
17	res += (waitPoints[i].middle(waitPoints[i-1])).distanceTo(waitPoints[i-1]);
18	}
19	return res;
20	}
21	public Location next() {
22	return currentIdx < waitPoints.length ? waitPoints[currentIdx++]
23	: waitPoints[waitPoints.length - 1];
24	}

1. Liste los métodos de instancia que implementaría la clase Location.
2. Liste los refactorings que aplicaría para eliminar el código repetido.
3. Transcriba el código resultante después de aplicar todos los refactorings.

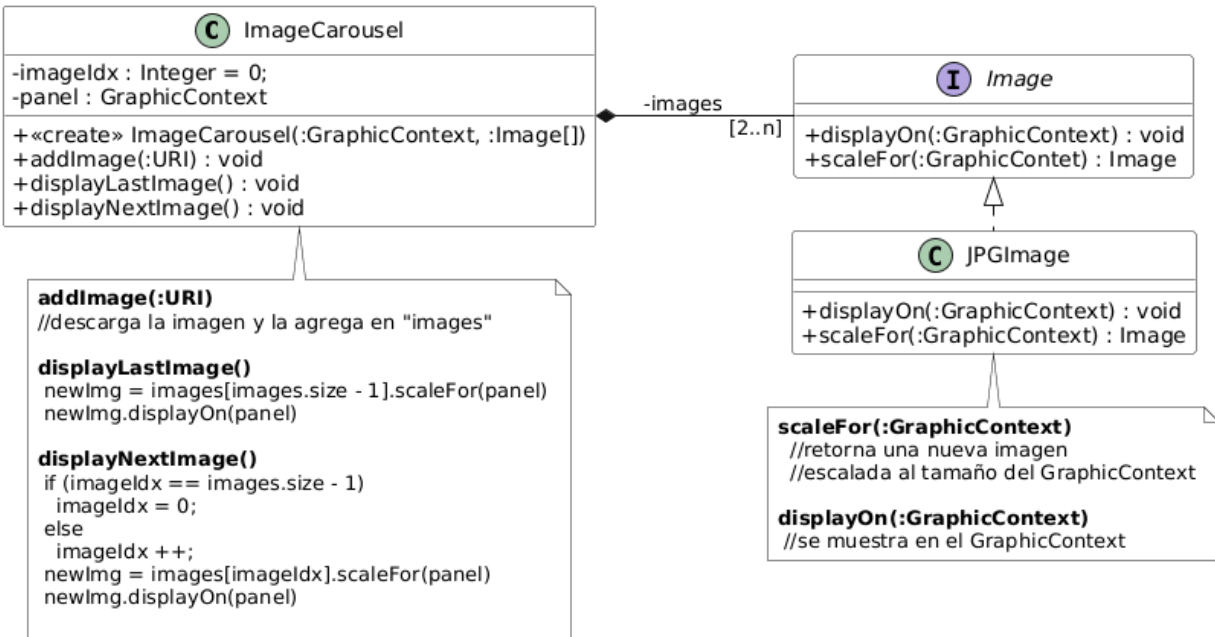
Ejercicio 2.

Un sistema de evaluación profesional usa un carrusel de fotos para que los usuarios encuentren elementos importantes en las imágenes. Cada evaluación cuenta con un conjunto determinado de imágenes. El carrusel siempre tiene al menos dos imágenes: una de inicio y otra de finalización. Las imágenes se presentan en secuencia y en cada una el participante evaluado debe realizar diferentes actividades. En algunos casos, como evaluaciones psicofísicas laborales, puede ocurrir que la evaluación termine antes de que el carrusel muestre todas las imágenes.

Un objeto Formulario maneja el carrusel y tiene el siguiente comportamiento:

- Inicializa el carrusel enviándole los URI de las imágenes (`addImage(:URI)`)
- Puede pedirle la siguiente imagen (`displayNextImage()`)
- Puede pedirle la última imagen (`displayLastImage()`).

Detalles de la evaluación o los contenidos de las imágenes están fuera del alcance de este enunciado.



La clase ImageCarousel tiene una lista ordenada de fotos que forman parte del estudio. Las imágenes son descargadas desde Internet, esto hace que la carga inicial del formulario sea muy lenta.

Los desarrolladores del carrusel planean aplicar el patrón Proxy, pero no están seguros de cómo hacerlo correctamente.

- Realice un diagrama de clases UML con la solución aplicando el patrón y presente el pseudocódigo correspondiente al diagrama.
- ¿Consideraría correcto aplicar el patrón State en lugar de Proxy? Justifique.

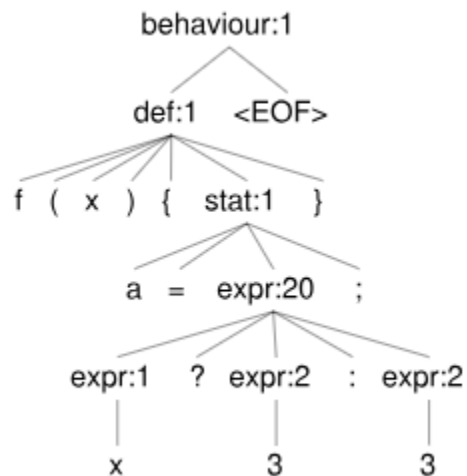
Ejercicio 3

Escriba un pseudocódigo que permita detectar el code smell indicado considerando el código fuente escrito en pool, y el árbol generado con antr lab

Considere que en el lenguaje dado por la cátedra, el llamado a una función o método retorna el valor generado por la evaluación de la última sentencia evaluada. (**stat**, en la gramática)

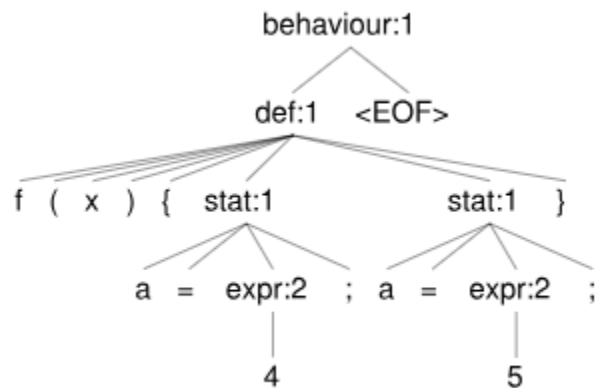
```
f(x) {
    a = x ? 3 : 3;
}
```

> Smell: ambas ramas del condicional son iguales.



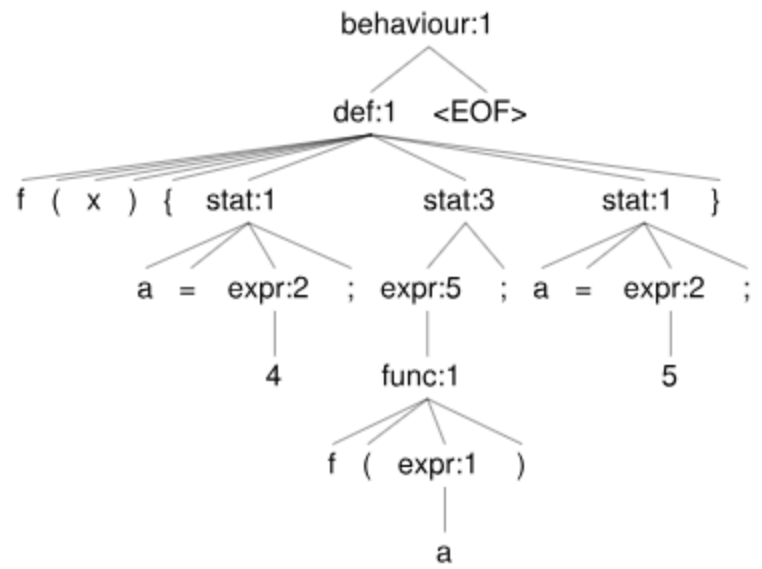
```
f(x) {
    a = 4;
    a = 5;
}
```

> Smell: La primera asignación no tiene efecto.



```
f(x) {
  a = 4;
  f(a);
  a = 5;
}
```

>Smell: parámetro "x" no es utilizado.



Ejercicio 4. Marque el patrón que aplicaría en cada caso.

1. Un mercado electrónico siempre se encuentra en uno de estos Modos durante su operación: Abierto, Cerrado, Suspendido. Se conocen las condiciones por las que un mercado cambia de un modo a otro.

Opciones: Strategy, Decorator, State, Builder.

2. Un sistema de mensajería permite agregar varias funcionalidades adicionales a los mensajes en tiempo de ejecución, como cifrado, compresión o firma digital.

Opciones: Strategy, Decorator, State, Builder.

3. Una fintech desarrolla una plataforma de pagos que necesita conectarse con los sistemas bancarios de distintos países. Cada banco implementa un conjunto estandarizado de operaciones, pero las interfaces son diferentes entre sí.

Opciones: Adapter, Decorator, State, Builder.

4. Un sistema de menú de hamburguesas que brinda al usuario opciones para poder formar su hamburguesa personalizada. El sistema permite elegir tipo de paty (de carne o de lentejas), ingredientes que acompañan al paty (tomate, cebolla, panceta, pepino) y un acompañamiento (papas fritas, aros de cebolla)

Opciones: Adapter, Decorator, State, Builder.